

How Java Stores Negative & Floating-Point Numbers

How Java Stores Negative & Floating-Point Numbers

1. Introduction
2. Storing Negative Numbers: Two's Complement
3. Storing Floating-Point Numbers: IEEE 754
4. What Is "Bias" and Why 127?
5. Why `0.7` Doesn't Print Exactly as `0.7`

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

How Java Stores Negative & Floating-Point Numbers

Two's Complement, IEEE 754, and Precision Issues — Lecture 4

1. Introduction

Positive integers are simple — take the number, convert to binary, store it, done. But two questions get asked **constantly in interviews**:

- How does Java store **negative** numbers using only 0s and 1s?
- How does Java store **decimal (floating-point)** numbers, and why does `0.7` sometimes print as `0.69999999...`?

This lecture answers both — with real, worked binary examples.

2. Storing Negative Numbers: Two's Complement

Java stores negative integers using a technique called **Two's Complement**.

Definition: Two's Complement is a method of representing negative numbers in binary such that the **Most Significant Bit (MSB)** alone tells you the sign — **1** = negative, **0** = positive — with no separate "negative zero" case to worry about.

MSB and LSB

Term	Full Form	Position	Meaning
MSB	Most Significant Bit	Left-most bit	Changing it shifts the value the <i>most</i>
LSB	Least Significant Bit	Right-most bit	Changing it shifts the value the <i>least</i>

💡 Easy Hinglish Explanation

"Jaise decimal mein 2321 ko agar last digit se +1 karein to 2322 ban jaata hai — thoda sa farak. Lekin agar first digit se +1 karein to 3321 ban jaata hai — bahut bada farak. Wahi 'significance' ka matlab hai — left-most bit (MSB) ka sabse zyada asar hota hai."

Worked Example: Storing `byte b = -42`

Step 1 — Find binary of the positive value (ignore the sign):

```
42 → 00101010 (8-bit byte)
```

Step 2 — Find the First Complement (invert every bit):

```
00101010 → 11010101
```

Step 3 — Find the Second Complement (add 1 to the first complement):

```

11010101
+       1
+-----
11010110    ← This is what gets stored in memory for -42

```

★ Exam Tip — Reading a stored number back

Check the **MSB**. If it's **1**, the computer knows the value is negative and reverses the two's-complement process to get the magnitude back. If it's **0**, it's simply read as positive.

Why not just flip the sign bit (One's Complement)?

⚠ **Common Mistake** Using only the **First Complement** (one's complement) creates **two representations of zero** — a "positive zero" (`00000000`) and a "negative zero" (`11111111`). This is messy for arithmetic and comparisons.

Two's complement fixes this: the second-complement of zero produces a 9th "overflow" bit that gets discarded, leaving a clean `00000000` — there's only ever **one zero**.

3. Storing Floating-Point Numbers: IEEE 754

Floating-point numbers (`float` , `double`) can't be stored the same simple way — they follow the **IEEE 754 standard**, which splits the bits into three sections:

Format	Total Bits	Sign	Exponent	Mantissa
float (single precision)	32-bit	1 bit	8 bits	23 bits
double (double precision)	64-bit	1 bit	11 bits	52 bits

Worked Example: Storing `float f = 8.125`

Step 1 — Convert to binary

```
8    → 1000
0.125 → 001    (multiply the fractional part by 2 repeatedly, read top-to-bottom)

8.125 → 1000.001
```

Step 2 — Normalize into the form `1.x × 2^exponent`

```
1000.001 → 1.000001 × 23
```

(Moved the decimal point 3 places left, so the power of 2 is 3.)

Step 3 — Add the bias to the exponent

```
exponent (3) + bias (127) = 130 → binary: 10000010
```

Step 4 — Place everything into the 32-bit layout

Section	Bits	Value
Sign	1 bit	0 (positive)
Exponent	8 bits	10000010
Mantissa	23 bits	0000010000000000000000000 (the x part after 1., padded with zeros)

 **Important:** The mantissa only stores the part *after* 1. — the leading 1 is never stored, because the format guarantees it's always there (this is called the "implicit leading bit").

4. What Is "Bias" and Why 127?

Definition: Bias is a fixed number **added to the exponent** so it's always stored as a positive (unsigned) number — even if the real exponent is negative. This makes hardware comparisons and calculations faster.

Formula:

$$\text{Bias} = 2^{(\text{exponent bits} - 1)} - 1$$

Type	Exponent Bits	Bias Formula	Bias Value
float	8	$2^7 - 1$	127
double	11	$2^{10} - 1$	1023

Easy Hinglish Explanation

"IEEE 754 standard kehta hai ki main koi bhi negative number store nahi karna chahta — isliye main exponent mein ek fixed 'bias' add kar deta hoon taaki wo hamesha positive rahe. Float ke liye ye bias 127 hai, double ke liye 1023 — kyunki dono ka exponent alag bits mein store hota hai."

5. Why 0.7 Doesn't Print Exactly as 0.7

Run this in Java:

```
float f = 0.7f;
System.out.printf("%.20f", f);
```

Output:

```
0.69999998807907104492
```

Why this happens: Some decimal fractions — like `0.7` — **cannot be represented exactly in binary**, no matter how many bits you use. The binary expansion repeats infinitely, just like `1/3` repeats infinitely in decimal (`0.3333...`).

Since `float` / `double` only have a **fixed number of mantissa bits**, the infinite binary sequence gets **truncated**, giving a value extremely close to — but not exactly — `0.7`.

⚠ Common Mistake

`System.out.println()` silently **rounds** the value for display, hiding this issue. Using `printf("%.20f", ...)` reveals the true stored value. Don't assume `println` shows you what's actually in memory.

 **Important** This is exactly why Java later introduces `BigDecimal` — a non-primitive type that stores decimal values **exactly**, with no precision loss. It's covered once non-primitive data types are introduced.

★ One-Page Revision**Core Definitions**

- **MSB** = left-most bit (highest significance); `1` = negative, `0` = positive
- **LSB** = right-most bit (lowest significance)
- **Two's Complement** = invert all bits (first complement) + add 1 (second complement)
- **Bias** = fixed value added to a floating-point exponent to keep it unsigned
- **Mantissa** = the fractional part after `1.` in the normalized `1.x × 2exp` form

Key Formulas

- Two's Complement: `First Complement (invert bits) + 1 = Second Complement`
- Bias = `2(exponent bits - 1) - 1`
- `float` bias = `127` · `double` bias = `1023`

Key Facts

- Two's complement avoids the "double zero" problem of one's complement
- `float` = 32 bits (1 sign + 8 exponent + 23 mantissa)
- `double` = 64 bits (1 sign + 11 exponent + 52 mantissa)

- Numbers like `0.7` can't be stored exactly in binary — the value gets truncated, causing tiny precision errors
- `BigDecimal` (a non-primitive type) solves this by storing decimals exactly



Practice Questions

Basic

1. What do MSB and LSB stand for, and how do they indicate sign?
2. What are the three sections of a 32-bit float, and how many bits does each get?

Medium 3. Manually compute the two's complement of `-15` as an 8-bit byte.

Interview 4. Why does Java use two's complement instead of one's complement for negative numbers? 5. Why does `0.7f` not print exactly as `0.7` in Java, even though `8.125` prints perfectly?



Quick FAQ

Is `double` immune to the precision issue? No — it just has more mantissa bits (52 vs 23), so it's *more precise*, not perfectly precise.

How do I get an exact decimal value in Java? Use `BigDecimal` instead of `float` / `double` when exact precision matters (e.g. financial calculations).